Chapter 14

Event-Driven Architecture Style

Event-Driven Architecture Style

The request-based model

- Most applications follow what is called a request-based model.
  - Requests made to the system to perform some sort of action are send to a request orchestrator.
  - The request orchestrator is typically a user interface, but it can also be implemented through an API layer or enterprise service bus.
  - The role of the request orchestrator is to deterministically and synchronously direct the request to various request processors. The request processors handle the request, either retrieving or updating information in a database.

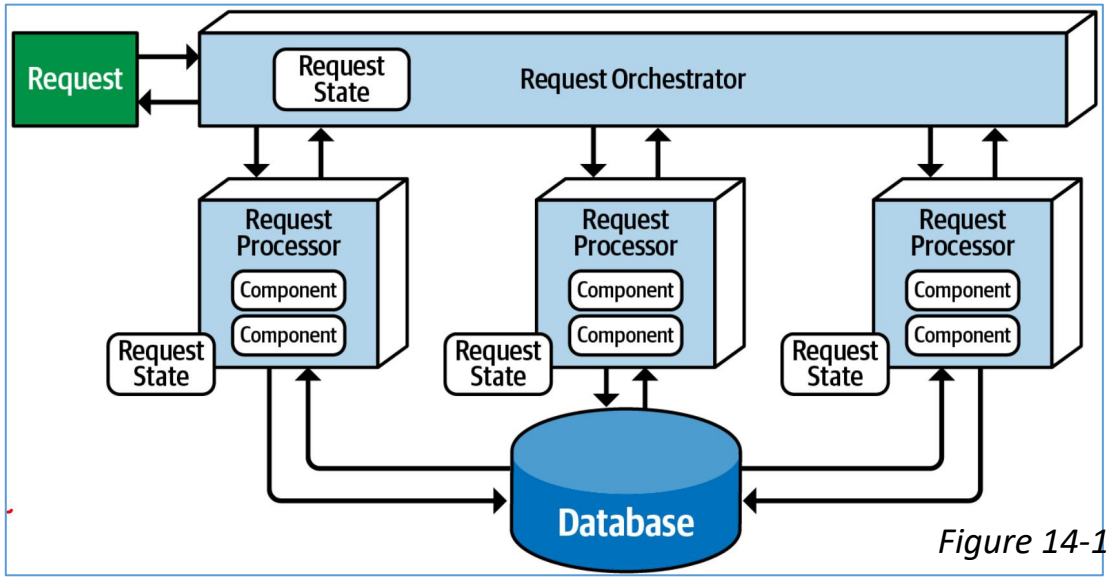


Figure 14-1. Request-based model

Event-Driven Architecture Style

Introduction

- An event-driven architecture uses events to trigger and communicate between decoupled services and is common in modern applications built with microservices. An event is a change in state, or an update, like an item being placed in a shopping cart on an e-commerce website.
- Event-driven architecture is made up of decoupled event processing components that asynchronously receive and process events. It can be used as a standalone architecture style or embedded within other architecture styles (such as an eventdriven microservices architecture).
- An event-based model, on the other hand, reacts to a particular situation and takes action based on that event.
  - An example of an event-based model is submitting a bid for a particular item within an online auction. Submitting the bid is not a request made to the system, but rather an event that happens after the current asking price is announced. The system must respond to this event by comparing the bid to others received at the same time to determine who is the current highest bidder.

Event-Driven Architecture Style

Topology

- There are two primary topologies within event-driven architecture:
  - the mediator topology and
  - the broker topology.
- The mediator topology is commonly used when you require control over the workflow of an event process, whereas the broker topology is used when you require a high degree of responsiveness and dynamic control over the processing of an event.
- Because the architecture characteristics and implementation strategies differ between these two topologies, it is important to understand each one to know which is best suited for a particular situation

Event-Driven Architecture Style

Broker Topology

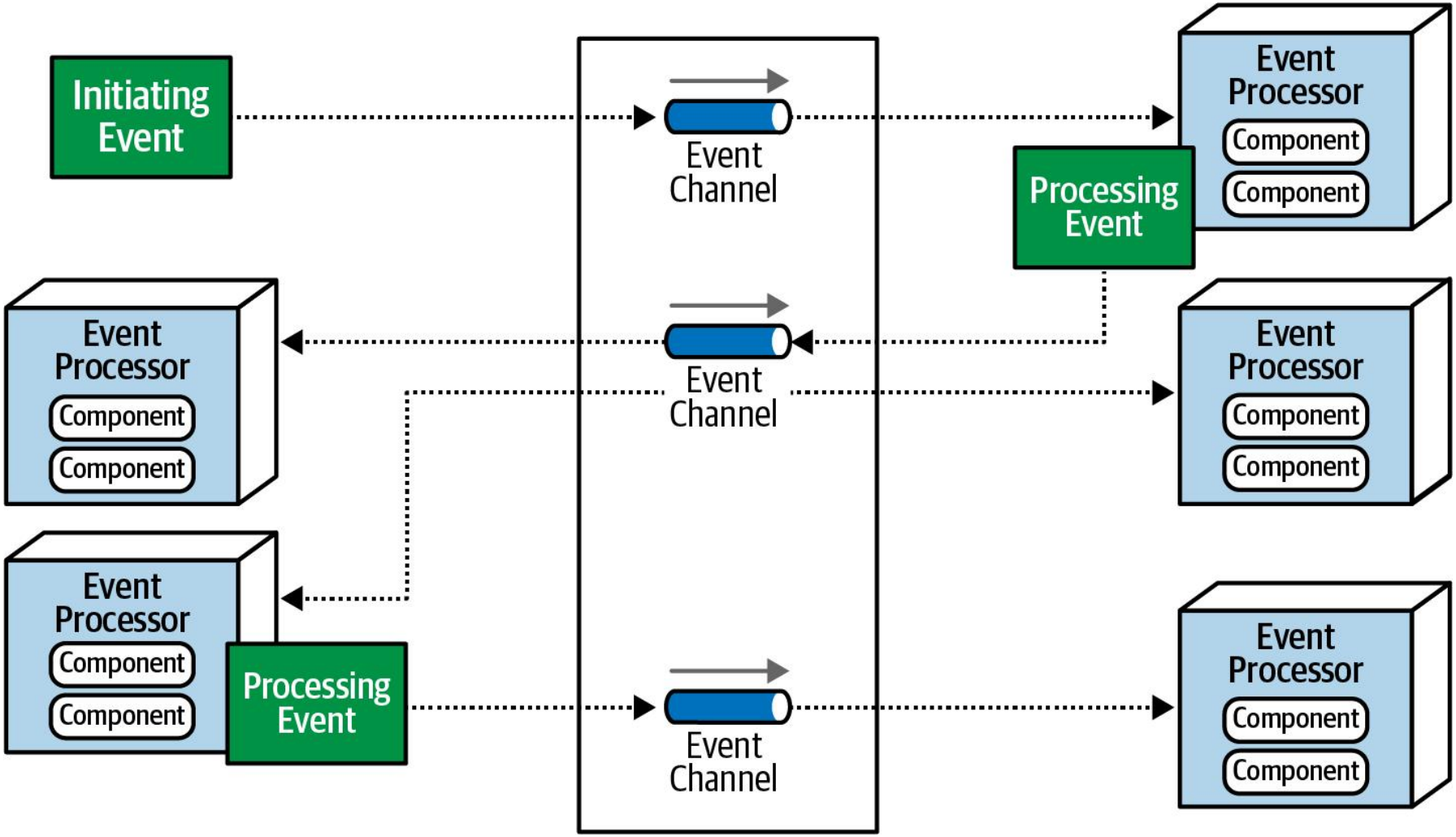


Figure 14-2. Broker topology

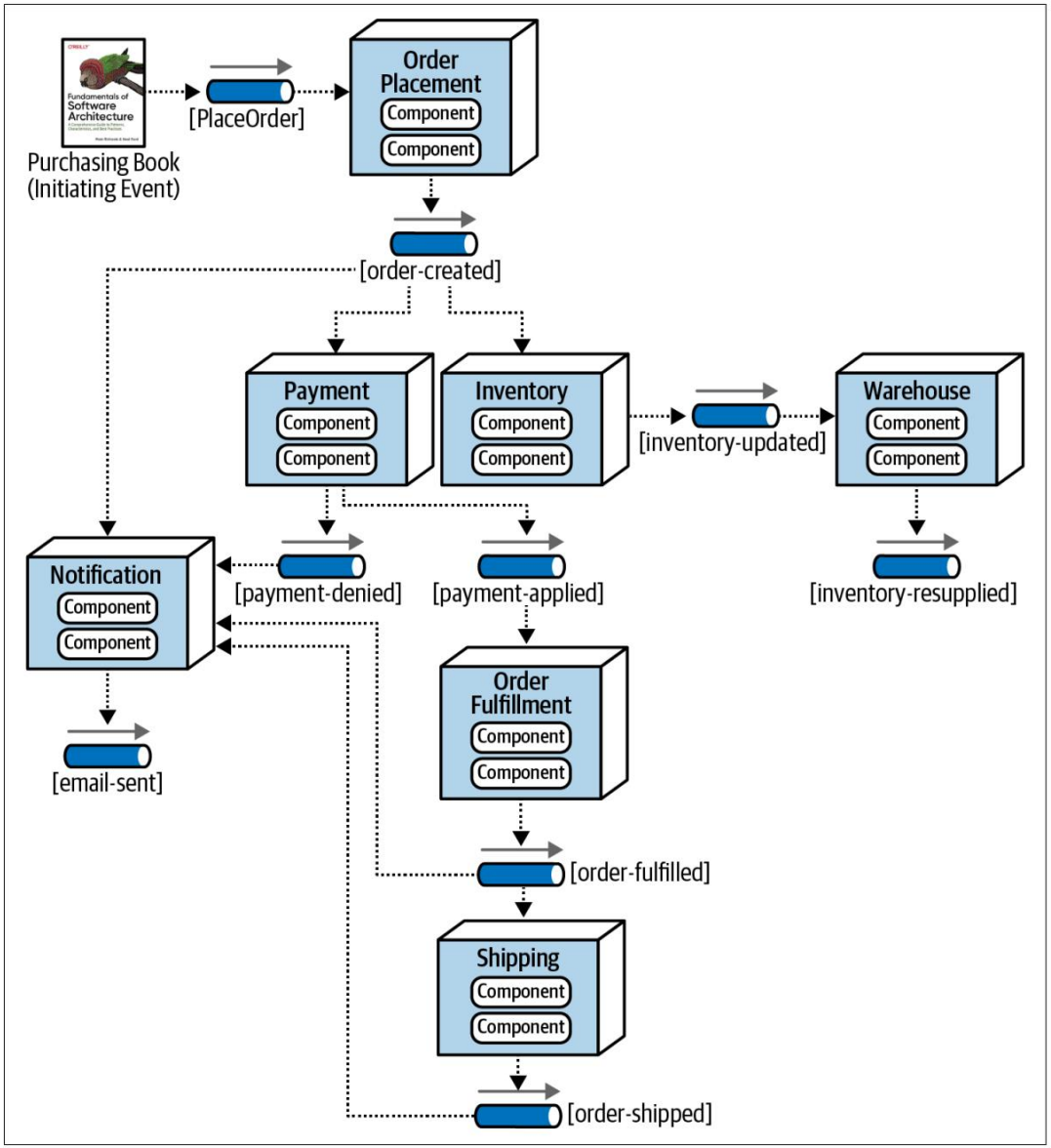
Event-Driven Architecture Style

Broker Topology

- There is no central event-mediator. Rather, the message flow is distributed across the event processor components in a chain-like broadcasting fashion through a lightweight message broker (such as RabbitMQ, ActiveMQ, HornetQ, and so on).
- This topology is useful when you have a relatively simple event processing flow and you do not need central event orchestration and coordination.
- The event broker component is usually federated (meaning multiple domain-based clustered instances), where each federated broker contains all of the event channels used within the event flow for that particular domain. Because of the decoupled asynchronous fire-and-forget broadcasting nature of the broker topology, topics (or topic exchanges in the case of AMQP) are usually used in the broker topology using a publish-and-subscribe messaging model.

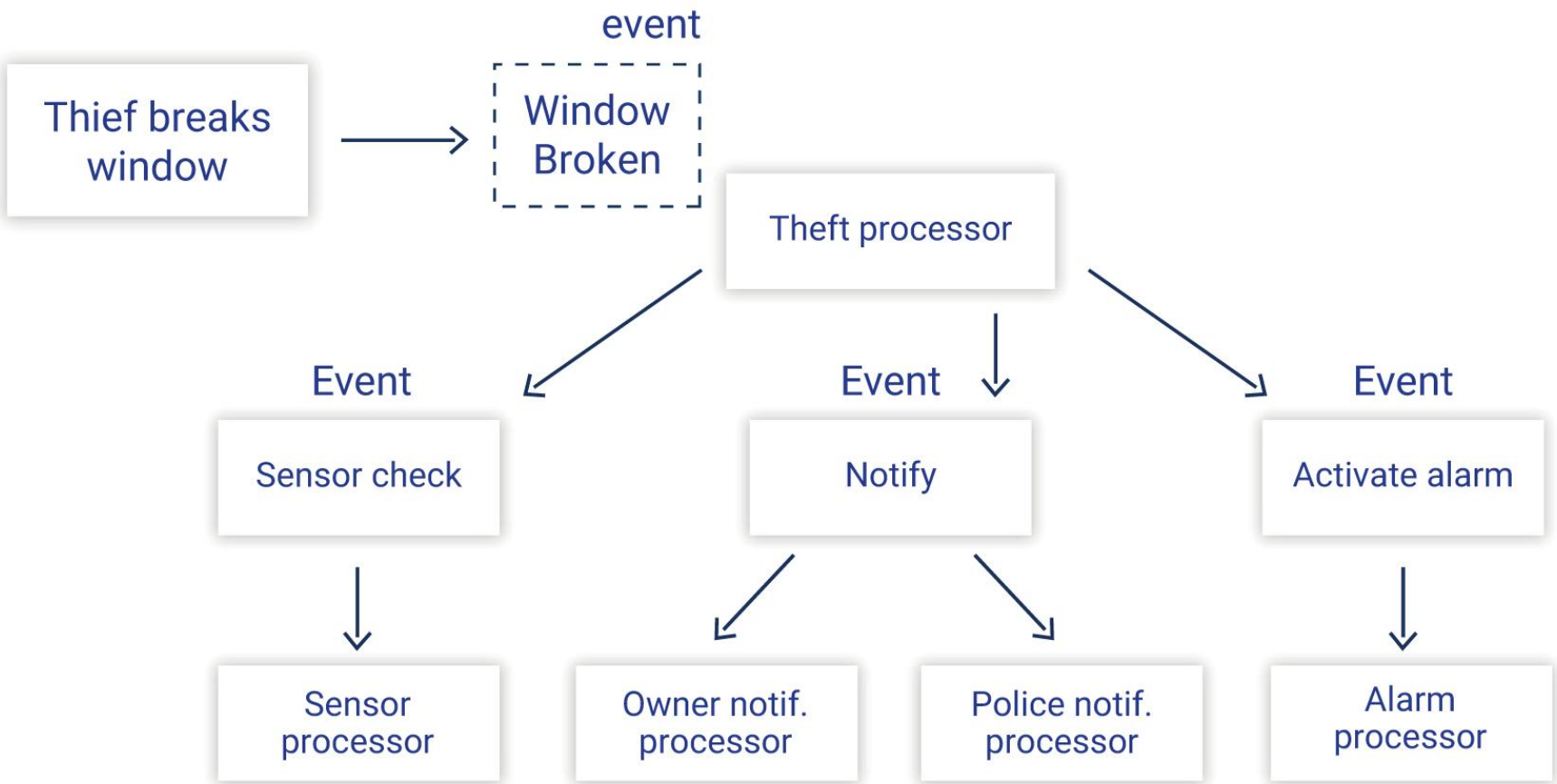
Event-Driven Architecture Style

Broker Topology - example



Event-Driven Architecture Style

Broker Topology - example (2)



Event-Driven Architecture Style

Broker Topology – Trade-off

Advantages	Disadvantages
Highly decoupled event processors	Workflow control
High scalability	Error handling
High responsiveness	Recoverability
High performance	Restart capabilities
High fault tolerance	Data inconsistency

Event-Driven Architecture Style

Mediator Topology

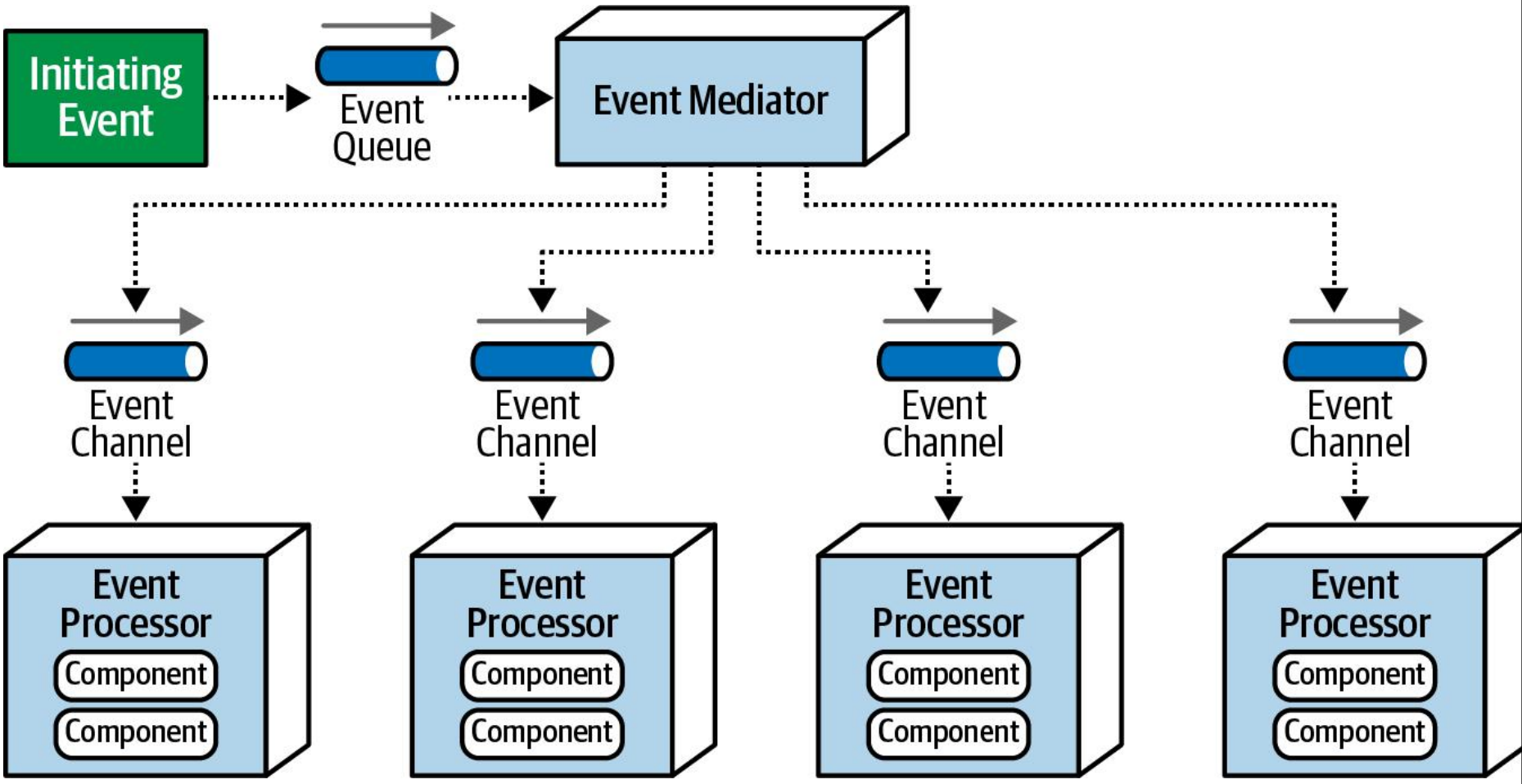


Figure 14-5. Mediator topology

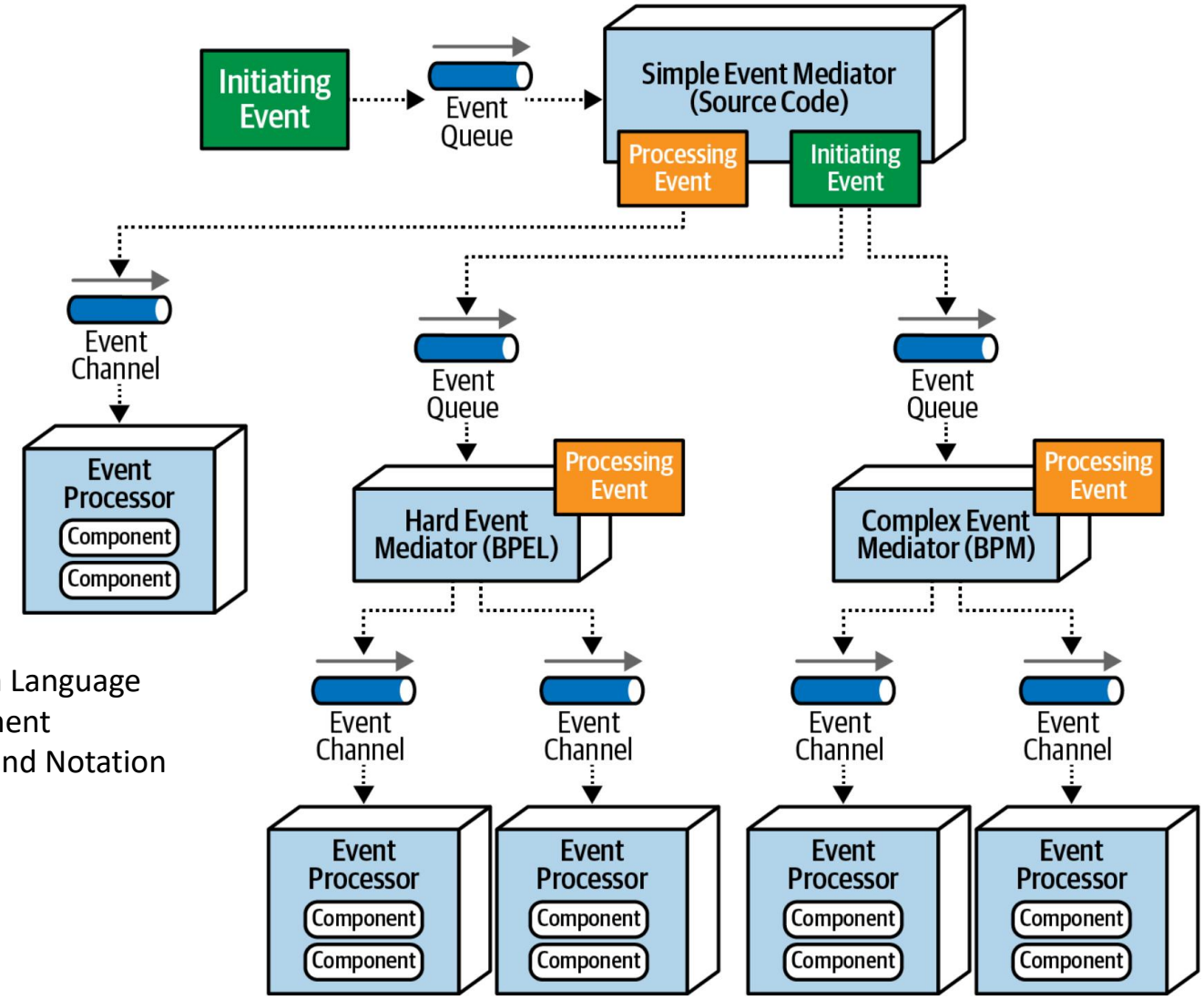
Event-Driven Architecture Style

Mediator Topology

- In most implementations of the mediator topology, there are multiple mediators, usually associated with a particular domain or grouping of events. This reduces the single point of failure issue associated with this topology and also increases overall throughput and performance.
  - For example, there might be a customer mediator that handles all customer-related events (such as new customer registration and profile update), and another mediator that handles order-related activities (such as adding an item to a shopping cart and checking out).
- The event mediator can be implemented in a variety of ways, depending on the nature and complexity of the events it is processing.
  - For example, for events requiring simple error handling and orchestration, a mediator such as Apache Camel, Mule ESB, or Spring Integration will usually suffice.
  - Message flows and message routes within these types of mediators are typically custom written in programming code (such as Java or C#) to control the workflow of the event processing.

Event-Driven Architecture Style

Mediator Topology



BPEL - Business Process Execution Language
BPM - Business Process Management
BPMN - Business Process Model and Notation

https://bpmn.io/
https://www.jbpm.org/

Figure 14-6. Delegating the event to the appropriate type of event mediator

Event-Driven Architecture Style

Mediator Topology – example

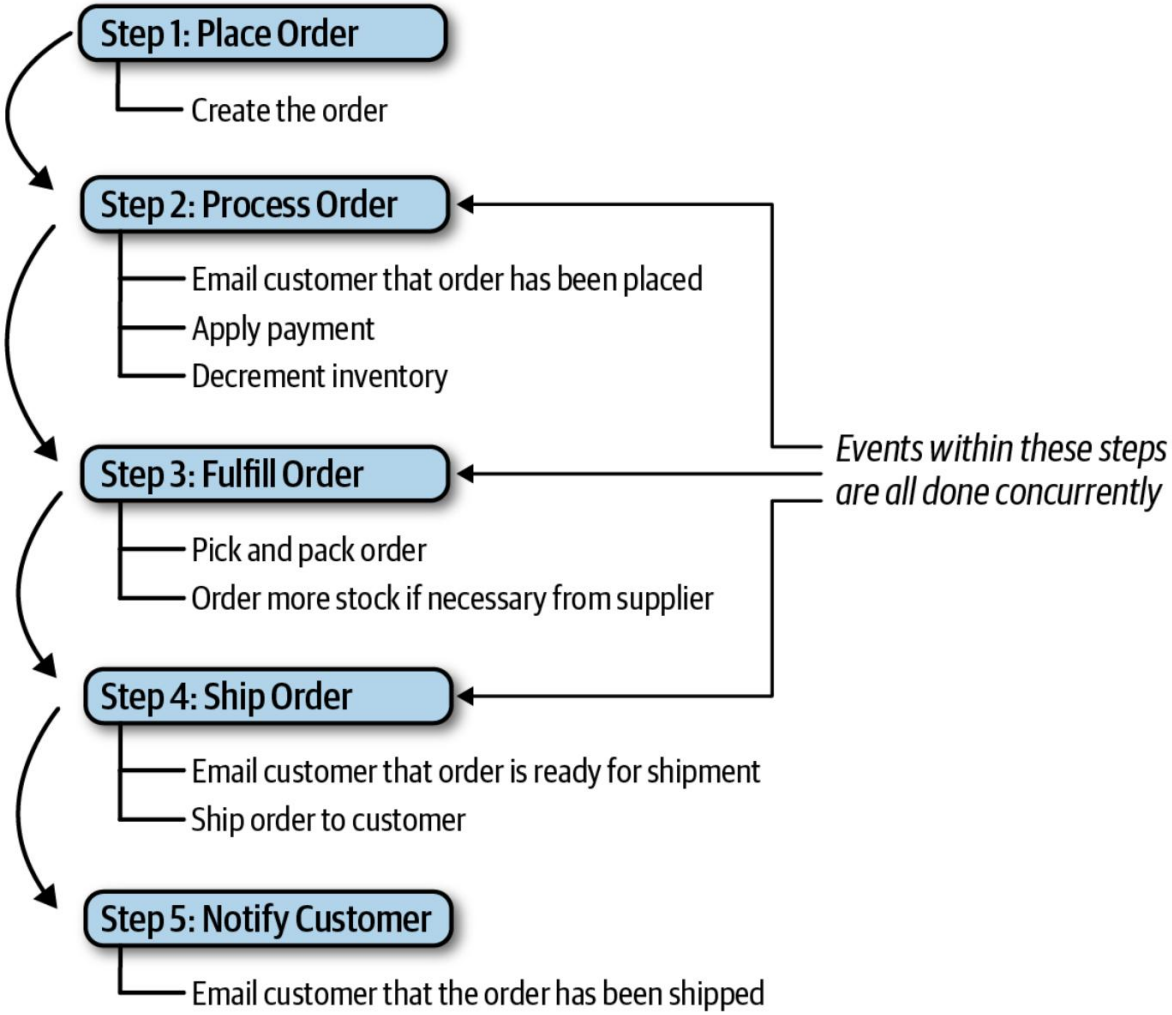
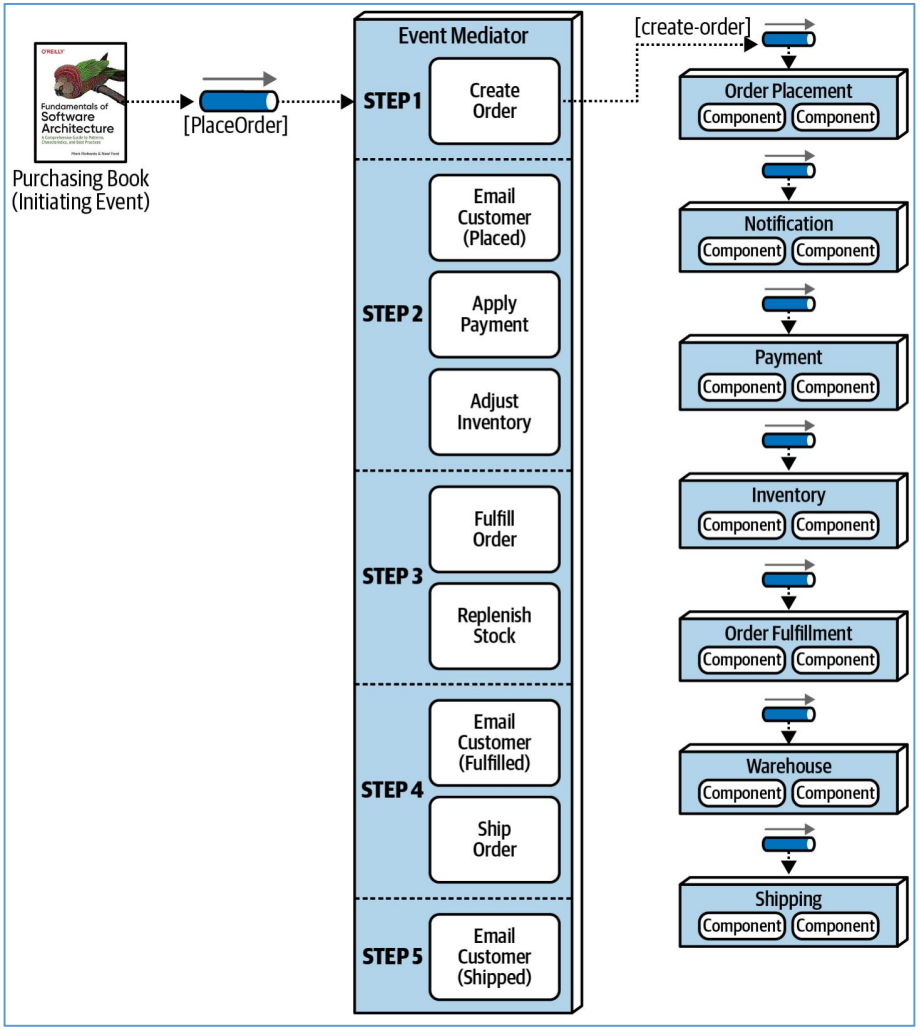
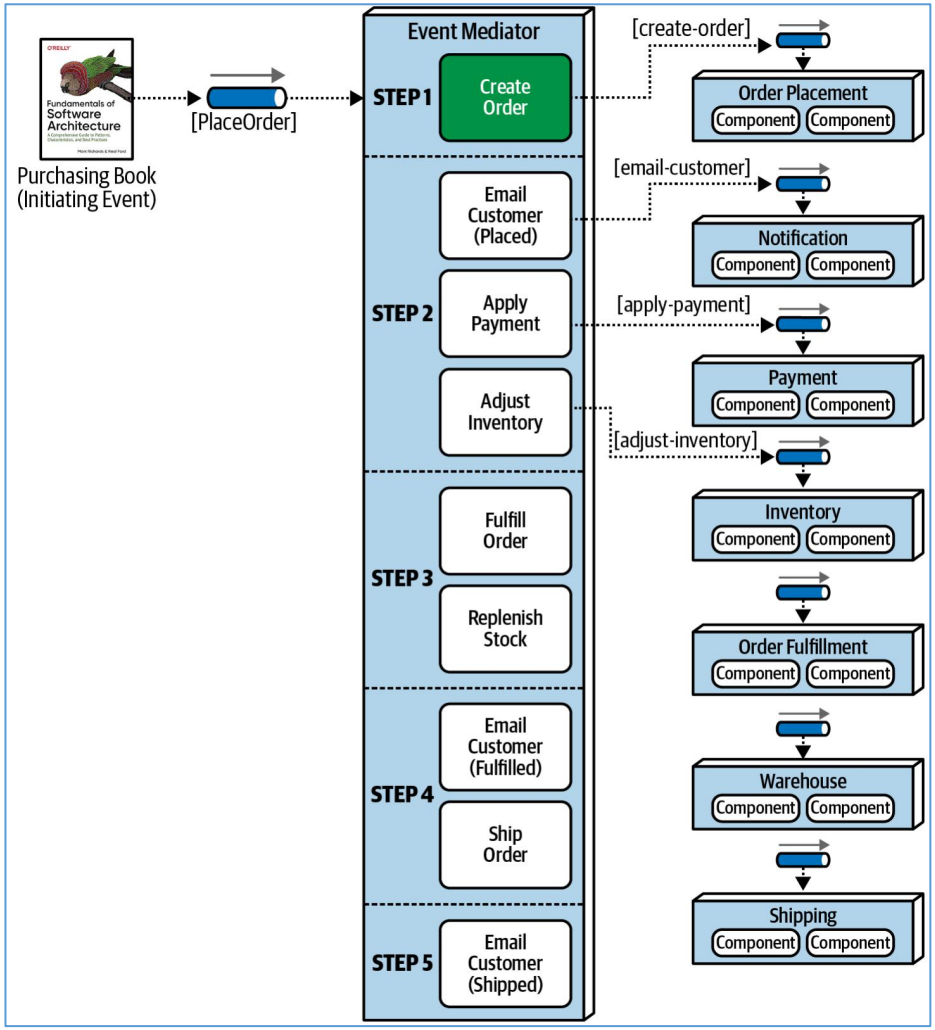


Figure 14-7. Mediator steps for placing an order

Event-Driven Architecture Style

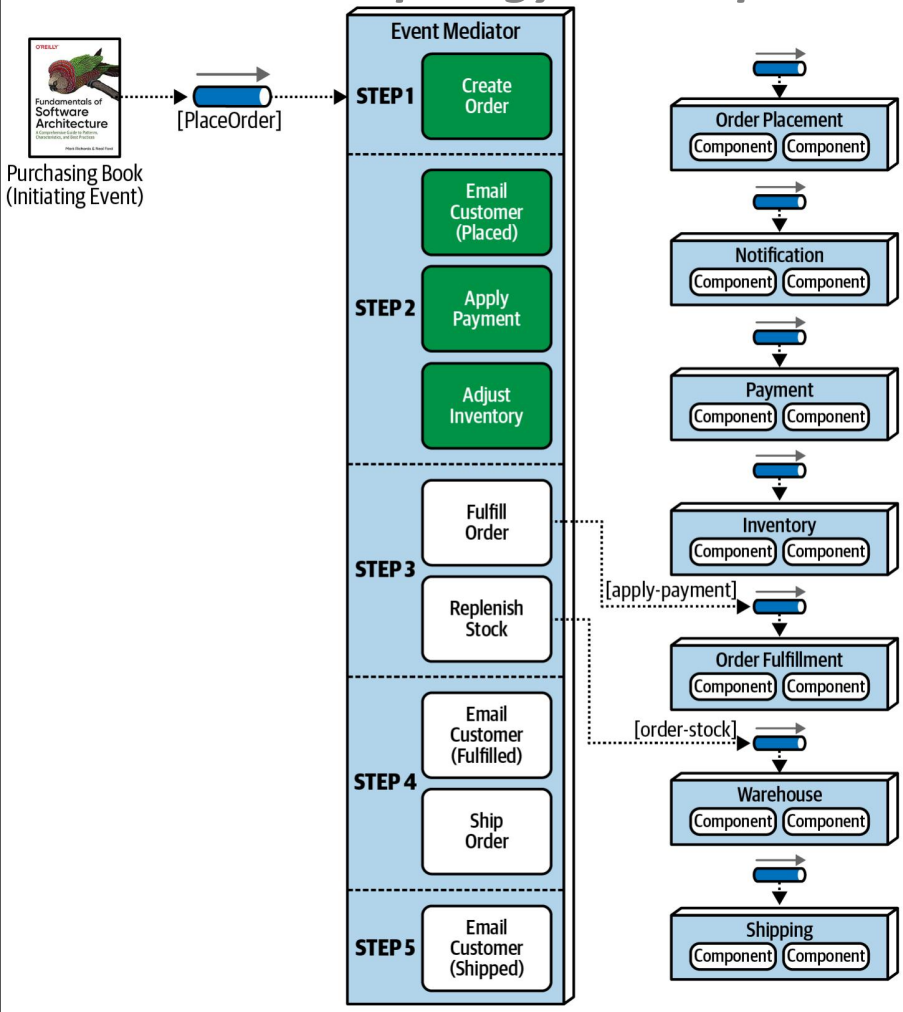
Mediator Topology - example

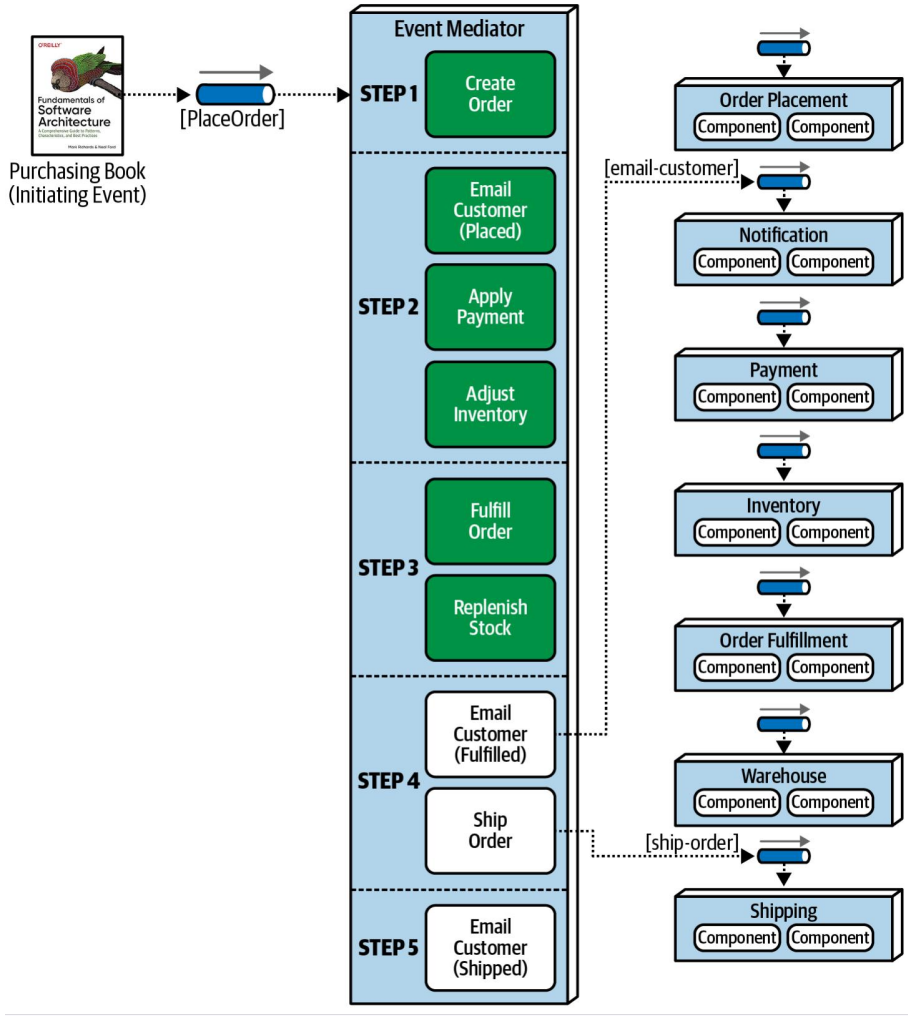




Event-Driven Architecture Style

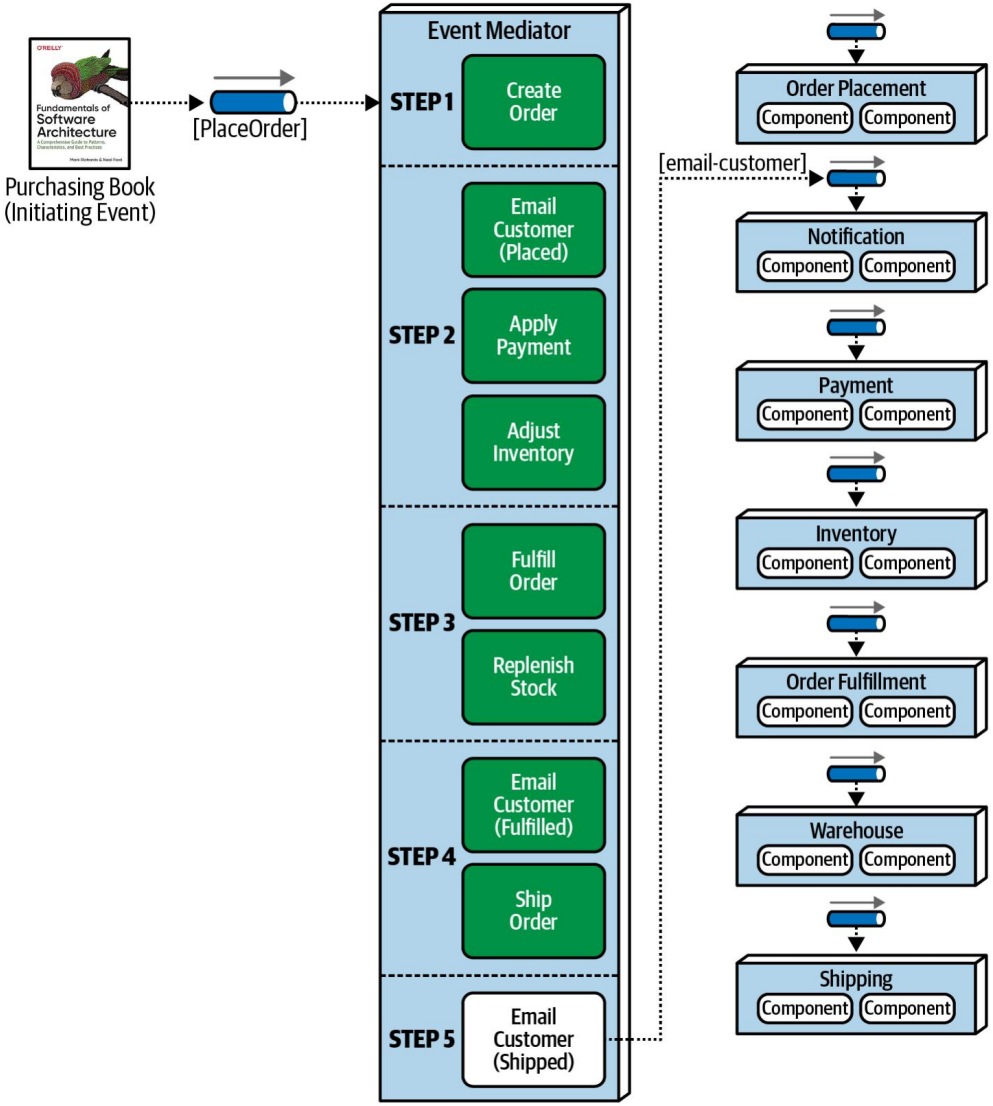
Mediator Topology - example





Event-Driven Architecture Style

Mediator Topology - example



Event-Driven Architecture Style

Mediator Topology – Trade-off

Advantages	Disadvantages
Workflow control	More coupling of event processors
Error handling	Lower scalability
Recoverability	Lower performance
Restart capabilities	Lower fault tolerance
Better data consistency	Modeling complex workflows

Event-Driven Architecture Style

Asynchronous Capabilities

- The event-driven architecture style offers a unique characteristic over other architecture styles in that it relies solely on asynchronous communication for both fire-and-forget processing (no response required) as well as request/reply processing (response required from the event consumer). Asynchronous communication can be a powerful technique for increasing the overall responsiveness of a system

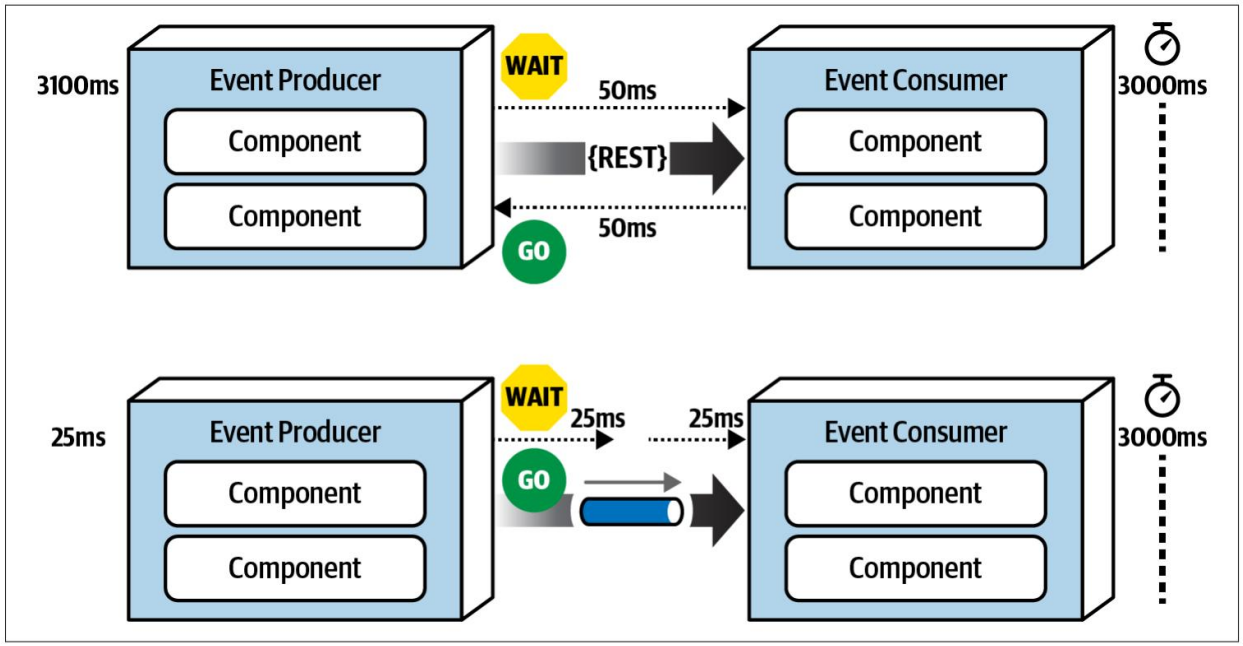


Figure 14-13. Synchronous versus asynchronous communication

Event-Driven Architecture Style

Error Handling

Event-Driven Architecture Style

Preventing Data Loss

Event-Driven Architecture Style

Broadcast Capabilities

Event-Driven Architecture Style

Request-Reply

Event-Driven Architecture Style

Choosing Between Request-Based and Event-Based

Event-Driven Architecture Style

Hybrid Event-Driven Architectures

Event-Driven Architecture Style

Architecture Characteristics Ratings

Event-Driven Architecture Style

Architecture Characteristics Ratings

Architecture characteristic	Star rating
Partitioning type	Technical
Number of quanta	1 to many
Deployability	★★★
Elasticity	★★★
Evolutionary	★★★★★
Fault tolerance	★★★★★
Modularity	★★★★
Overall cost	★★★
Performance	★★★★★
Reliability	★★★
Scalability	★★★★★
Simplicity	★
Testability	★★